Guide Complet des Pthreads

Programmation Concurrente en C

Document de Révision pour Examen

10 novembre 2025

Table des matières

1 Introduction aux Pthreads **3**
1.1 Différence entre Processus et Thread . . . 3
1.2 Avantages des Pthreads . . . 3

2 Création et Gestion des Threads **3**
2.1 Fonction pthread_create() . . . 3
2.2 Exemple 1 : Création Simple d'un Thread . . . 4
2.3 Fonction pthread_join() . . . 5
2.4 Exemple 2 : Threads Multiples avec Valeurs de Retour . . . 5
2.5 Fonction pthread_exit() . . . 6
2.6 Fonction pthread_detach() . . . 6

3 Synchronisation : Mutex **7**
3.1 Problème de la Section Critique . . . 7
3.2 Fonctions Mutex Principales . . . 7
3.3 Exemple 3 : Compteur Partagé SANS Mutex (Problème) . . . 7
3.4 Exemple 4 : Compteur Partagé AVEC Mutex (Solution) . . . 8
3.5 Exemple 5 : Compte Bancaire Partagé . . . 8

4 Synchronisation : Variables de Condition **10**
4.1 Fonctions Principales . . . 10
4.2 Exemple 6 : Producteur-Consommateur . . . 11

5 Attributs des Threads **13**
5.1 Initialisation et Destruction . . . 13
5.2 Attributs Principaux . . . 13

6 Sémaphores POSIX **13**
6.1 Fonctions Principales . . . 13
6.2 Exemple 7 : Pool de Connexions . . . 14

7 Barrières de Synchronisation **15**
7.1 Exemple 8 : Calcul Parallèle par Phases . . . 15

8 Thread-Local Storage (TLS) **16**
8.1 Utilisation avec __thread . . . 16

9 Gestion des Erreurs **17**
9.1 Codes d'Erreur Courants . . . 17
9.2 Exemple de Gestion d'Erreurs . . . 17

10 Problèmes Classiques et Solutions **17**
10.1 Deadlock (Interblocage) . . . 18
10.2 Exemple de Deadlock . . . 18
10.3 Solution : Ordre de Verrouillage . . . 18

11 Bonnes Pratiques **19**

12 Résumé des Fonctions Principales **19**
12.1 Tableau Récapitulatif . . . 20

13 Exercices Types pour l'Examen **20**
13.1 Exercice 1 : Analyse de Code 20
13.2 Exercice 2 : Implémentation 21
13.3 Exercice 3 : Synchronisation 21

14 Conclusion **21**

1 Introduction aux Pthreads

Qu'est-ce qu'un Thread ?

Un **thread** (fil d'exécution) est une unité d'exécution légère au sein d'un processus. Plusieurs threads peuvent s'exécuter simultanément dans le même espace mémoire, partageant les ressources du processus.

1.1 Différence entre Processus et Thread

Processus	Thread
Espace mémoire séparé	Partage l'espace mémoire
Création coûteuse	Création légère
Communication complexe (IPC)	Communication simple (variables partagées)
Isolation forte	Isolation faible

1.2 Avantages des Pthreads

— **Performance** : Exploitation du parallélisme sur processeurs multi-cœurs
— **Réactivité** : Interface utilisateur reste responsive pendant calculs lourds
— **Partage de ressources** : Communication efficace entre threads
— **Économie** : Moins coûteux que la création de processus

Compilation avec Pthreads

Pour compiler un programme utilisant pthreads :

```
gcc -pthread programme.c -o programme
```

L'option `-pthread` est **obligatoire** !

2 Création et Gestion des Threads

2.1 Fonction pthread_create()

Prototype

```
int pthread_create(pthread_t *thread,
                   const pthread_attr_t *attr,
                   void *(*start_routine)(void *),
                   void *arg);
```

Paramètres :

— `thread` : Pointeur vers l'identifiant du thread créé
— `attr` : Attributs du thread (NULL pour valeurs par défaut)
— `start_routine` : Fonction que le thread va exécuter
— `arg` : Argument passé à la fonction

Retour : 0 en cas de succès, code d'erreur sinon

2.2 Exemple 1 : Création Simple d'un Thread

```c
#include <stdio.h>          // Entrées/sorties standard
#include <stdlib.h>         // Fonctions utilitaires
#include <pthread.h>        // Bibliothèque pthreads
#include <unistd.h>         // Pour sleep()

// Fonction exécutée par le thread
void* fonction_thread(void* arg) {
    // Conversion de l'argument void* en int*
    int id = *(int*)arg;

    // Affichage de l'ID du thread
    printf("Thread %d : D marrage\n", id);

    // Simulation d'un travail (2 secondes)
    sleep(2);

    // Message de fin
    printf("Thread %d : Terminaison\n", id);

    // Retour NULL (pas de valeur de retour)
    return NULL;
}

int main() {
    // Déclaration de l'identifiant du thread
    pthread_t thread;

    // ID à passer au thread
    int id = 1;

    // Création du thread
    // - &thread : où stocker l'ID du thread
    // - NULL : attributs par défaut
    // - fonction_thread : fonction à exécuter
    // - &id : argument passé à la fonction
    if (pthread_create(&thread, NULL, fonction_thread, &id) != 0) {
        perror("Erreur pthread_create");
        return 1;
    }

    // Le thread principal continue son exécution
    printf("Main : Thread cr    avec succ s\n");

    // Attendre la fin du thread
    pthread_join(thread, NULL);

    // Fin du programme
    printf("Main : Programme termin \n");
    return 0;
}
```

Listing 1 – Création basique d'un thread

2.3 Fonction pthread_join()

Prototype

```
int pthread_join(pthread_t thread, void **retval);
```

Rôle : Attend la terminaison d'un thread spécifique (équivalent de `wait()` pour les processus)
Paramètres :

— `thread` : Identifiant du thread à attendre

— `retval` : Pointeur pour récupérer la valeur de retour (NULL si non nécessaire)

2.4 Exemple 2 : Threads Multiples avec Valeurs de Retour

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

// Fonction qui calcule le carré d'un nombre
void* calculer_carre(void* arg) {
    // Récupération du nombre
    int num = *(int*)arg;

    // Allocation dynamique pour le résultat
    // (nécessaire car la variable locale disparaît)
    int* resultat = malloc(sizeof(int));

    // Calcul du carré
    *resultat = num * num;

    printf("Thread : %d^2 = %d\n", num, *resultat);

    // Retour du résultat (cast en void*)
    return (void*)resultat;
}

int main() {
    // Tableau de 5 threads
    pthread_t threads[5];
    // Tableau des nombres à traiter
    int nombres[5] = {1, 2, 3, 4, 5};

    // Création des 5 threads
    for (int i = 0; i < 5; i++) {
        if (pthread_create(&threads[i], NULL, calculer_carre,
            &nombres[i]) != 0) {
            fprintf(stderr, "Erreur cr ation thread %d\n", i);
            return 1;
        }
    }

    // Attente et récupération des résultats
    for (int i = 0; i < 5; i++) {
        // Pointeur pour récupérer la valeur de retour
        void* retval;

        // Attente du thread i
        pthread_join(threads[i], &retval);

        // Conversion et affichage du résultat
```

```
46          int* resultat = (int*)retval;
47          printf("Main : R sultat thread %d = %d\n", i, *resultat);
48
49          // Libération de la mémoire allouée par le thread
50          free(resultat);
51      }
52
53      return 0;
54  }
```

Listing 2 – Création de plusieurs threads avec retour de valeurs

2.5 Fonction pthread_exit()

Prototype

```
void pthread_exit(void *retval);
```

Rôle : Termine le thread appelant et retourne une valeur

Différence return vs pthread_exit()

- `return` : Termine uniquement la fonction du thread
- `pthread_exit()` : Termine le thread immédiatement, même depuis une fonction appelée

2.6 Fonction pthread_detach()

Prototype

```
int pthread_detach(pthread_t thread);
```

Rôle : Détache un thread, permettant la libération automatique de ses ressources à sa terminaison (pas besoin de `pthread_join()`)

```
1  void* fonction_detachee(void* arg) {
2      // Se détacher soi-même
3      pthread_detach(pthread_self());
4
5      // Travail du thread
6      printf("Thread d tach  en cours d'ex cution\n");
7      sleep(1);
8
9      return NULL;
10 }
11
12 int main() {
13     pthread_t thread;
14     pthread_create(&thread, NULL, fonction_detachee, NULL);
15
16     // Pas besoin de pthread_join()
17     // Le thread se nettoie automatiquement
18
19     sleep(2); // Attendre que le thread finisse
20     return 0;
21 }
```

Listing 3 – Utilisation de pthread_detach()

3 Synchronisation : Mutex

Qu'est-ce qu'un Mutex ?

Un **mutex** (Mutual Exclusion) est un verrou qui garantit qu'un seul thread à la fois peut accéder à une section critique (ressource partagée).

3.1 Problème de la Section Critique

Race Condition

Quand plusieurs threads accèdent simultanément à une variable partagée sans synchronisation, le résultat est **imprévisible** !

3.2 Fonctions Mutex Principales

Déclaration et Initialisation

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
// ou
pthread_mutex_t mutex;
pthread_mutex_init(&mutex, NULL);
```

Fonctions de base :

- `pthread_mutex_lock()` : Verrouille le mutex (attend s'il est déjà verrouillé)
- `pthread_mutex_unlock()` : Déverrouille le mutex
- `pthread_mutex_trylock()` : Tente de verrouiller sans bloquer
- `pthread_mutex_destroy()` : Détruit le mutex

3.3 Exemple 3 : Compteur Partagé SANS Mutex (Problème)

```c
#include <stdio.h>
#include <pthread.h>

// Variable partagée (DANGER sans protection !)
int compteur = 0;

void* incrementer(void* arg) {
    for (int i = 0; i < 1000000; i++) {
        // SECTION CRITIQUE NON PROTÉGÉE
        // Opération non atomique : lecture, incrémentation, écriture
        compteur++;  // RACE CONDITION !
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, incrementer, NULL);
    pthread_create(&t2, NULL, incrementer, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

```

```
    // Résultat attendu : 2000000
    // Résultat réel : IMPRÉVISIBLE (souvent < 2000000)
    printf("Compteur final : %d\n", compteur);

    return 0;
}
```

Listing 4 – Problème de race condition

3.4 Exemple 4 : Compteur Partagé AVEC Mutex (Solution)

```
#include <stdio.h>
#include <pthread.h>

// Variable partagée
int compteur = 0;

// Mutex pour protéger la section critique
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void* incrementer(void* arg) {
    for (int i = 0; i < 1000000; i++) {
        // VERROUILLAGE : début de la section critique
        pthread_mutex_lock(&mutex);

        // Section critique protégée
        // Un seul thread à la fois peut exécuter ce code
        compteur++;

        // DÉVERROUILLAGE : fin de la section critique
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, incrementer, NULL);
    pthread_create(&t2, NULL, incrementer, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    // Résultat garanti : 2000000
    printf("Compteur final : %d\n", compteur);

    // Destruction du mutex
    pthread_mutex_destroy(&mutex);

    return 0;
}
```

Listing 5 – Protection avec mutex

3.5 Exemple 5 : Compte Bancaire Partagé

```
#include <stdio.h>
#include <stdlib.h>
```

```c
#include <pthread.h>
#include <unistd.h>

// Structure représentant un compte bancaire
typedef struct {
    int solde;                    // Solde du compte
    pthread_mutex_t mutex;        // Mutex pour protéger le solde
} CompteBancaire;

// Structure pour passer les arguments aux threads
typedef struct {
    CompteBancaire* compte;
    int montant;
    int id_thread;
} ArgsTransaction;

// Fonction de dépôt
void* deposer(void* arg) {
    ArgsTransaction* args = (ArgsTransaction*)arg;

    // Verrouillage du compte
    pthread_mutex_lock(&args->compte->mutex);

    // Lecture du solde actuel
    int solde_avant = args->compte->solde;

    // Simulation d'un délai de traitement
    usleep(100);

    // Ajout du montant
    args->compte->solde += args->montant;

    printf("Thread %d : D  p  t  de % d   (Solde : % d   -> % d  )\n",
           args->id_thread, args->montant,
           solde_avant, args->compte->solde);

    // Déverrouillage du compte
    pthread_mutex_unlock(&args->compte->mutex);

    free(args);
    return NULL;
}

// Fonction de retrait
void* retirer(void* arg) {
    ArgsTransaction* args = (ArgsTransaction*)arg;

    pthread_mutex_lock(&args->compte->mutex);

    // Vérification du solde suffisant
    if (args->compte->solde >= args->montant) {
        int solde_avant = args->compte->solde;
        usleep(100);
        args->compte->solde -= args->montant;

        printf("Thread %d : Retrait de % d   (Solde : % d   -> % d  )\n",
               args->id_thread, args->montant,
               solde_avant, args->compte->solde);
    } else {
        printf("Thread %d : Retrait refus  (solde insuffisant)\n",
               args->id_thread);
```

```c
    }

    pthread_mutex_unlock(&args->compte->mutex);

    free(args);
    return NULL;
}

int main() {
    // Initialisation du compte avec 1000€
    CompteBancaire compte;
    compte.solde = 1000;
    pthread_mutex_init(&compte.mutex, NULL);

    // Création de 10 threads (5 dépôts, 5 retraits)
    pthread_t threads[10];

    for (int i = 0; i < 5; i++) {
        // Thread de dépôt
        ArgsTransaction* args_depot = malloc(sizeof(ArgsTransaction));
        args_depot->compte = &compte;
        args_depot->montant = 100;
        args_depot->id_thread = i;
        pthread_create(&threads[i], NULL, deposer, args_depot);

        // Thread de retrait
        ArgsTransaction* args_retrait = malloc(sizeof(ArgsTransaction));
        args_retrait->compte = &compte;
        args_retrait->montant = 150;
        args_retrait->id_thread = i + 5;
        pthread_create(&threads[i+5], NULL, retirer, args_retrait);
    }

    // Attente de tous les threads
    for (int i = 0; i < 10; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("\nSolde final : %d  \n", compte.solde);

    // Nettoyage
    pthread_mutex_destroy(&compte.mutex);

    return 0;
}
```

Listing 6 – Simulation de transactions bancaires

4 Synchronisation : Variables de Condition

Qu'est-ce qu'une Variable de Condition ?

Une **variable de condition** permet à un thread d'attendre qu'une certaine condition soit vraie avant de continuer. Elle est toujours utilisée avec un mutex.

4.1 Fonctions Principales

— `pthread_cond_init()` : Initialise une variable de condition

— `pthread_cond_wait()` : Attend un signal (libère le mutex pendant l'attente)
— `pthread_cond_signal()` : Réveille UN thread en attente
— `pthread_cond_broadcast()` : Réveille TOUS les threads en attente
— `pthread_cond_destroy()` : Détruit la variable de condition

4.2 Exemple 6 : Producteur-Consommateur

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define TAILLE_BUFFER 5

// Buffer partagé
int buffer[TAILLE_BUFFER];
int compteur = 0;  // Nombre d'éléments dans le buffer

// Mutex et variables de condition
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t buffer_non_plein = PTHREAD_COND_INITIALIZER;
pthread_cond_t buffer_non_vide = PTHREAD_COND_INITIALIZER;

// Fonction producteur
void* producteur(void* arg) {
    int id = *(int*)arg;

    for (int i = 0; i < 10; i++) {
        // Production d'un élément
        int item = rand() % 100;

        // Verrouillage du mutex
        pthread_mutex_lock(&mutex);

        // Attendre que le buffer ne soit pas plein
        while (compteur == TAILLE_BUFFER) {
            printf("Producteur %d : Buffer plein, attente...\n", id);
            // pthread_cond_wait libère le mutex et attend
            // Quand réveillé, il re-verrouille automatiquement le mutex
            pthread_cond_wait(&buffer_non_plein, &mutex);
        }

        // Ajout de l'élément dans le buffer
        buffer[compteur] = item;
        compteur++;

        printf("Producteur %d : Produit %d (buffer: %d/%d)\n",
               id, item, compteur, TAILLE_BUFFER);

        // Signaler qu'il y a un nouvel élément
        pthread_cond_signal(&buffer_non_vide);

        // Déverrouillage du mutex
        pthread_mutex_unlock(&mutex);

        // Pause aléatoire
        usleep(rand() % 500000);
    }

```

```c
    return NULL;
}

// Fonction consommateur
void* consommateur(void* arg) {
    int id = *(int*)arg;

    for (int i = 0; i < 10; i++) {
        pthread_mutex_lock(&mutex);

        // Attendre que le buffer ne soit pas vide
        while (compteur == 0) {
            printf("Consommateur %d : Buffer vide, attente...\n", id);
            pthread_cond_wait(&buffer_non_vide, &mutex);
        }

        // Retrait d'un élément du buffer
        compteur--;
        int item = buffer[compteur];

        printf("Consommateur %d : Consomm  %d (buffer: %d/%d)\n",
               id, item, compteur, TAILLE_BUFFER);

        // Signaler qu'il y a de la place
        pthread_cond_signal(&buffer_non_plein);

        pthread_mutex_unlock(&mutex);

        usleep(rand() % 500000);
    }

    return NULL;
}

int main() {
    pthread_t prod[2], cons[2];
    int ids[2] = {1, 2};

    // Création de 2 producteurs et 2 consommateurs
    for (int i = 0; i < 2; i++) {
        pthread_create(&prod[i], NULL, producteur, &ids[i]);
        pthread_create(&cons[i], NULL, consommateur, &ids[i]);
    }

    // Attente de tous les threads
    for (int i = 0; i < 2; i++) {
        pthread_join(prod[i], NULL);
        pthread_join(cons[i], NULL);
    }

    // Nettoyage
    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&buffer_non_plein);
    pthread_cond_destroy(&buffer_non_vide);

    return 0;
}
```

Listing 7 – Problème classique producteur-consommateur

5 Attributs des Threads

5.1 Initialisation et Destruction

```c
#include <pthread.h>

pthread_attr_t attr;

// Initialisation des attributs
pthread_attr_init(&attr);

// Configuration des attributs
pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);

// Création du thread avec attributs
pthread_t thread;
pthread_create(&thread, &attr, fonction, NULL);

// Destruction des attributs
pthread_attr_destroy(&attr);
```

Listing 8 – Utilisation des attributs

5.2 Attributs Principaux

- **Detach State** : `PTHREAD_CREATE_JOINABLE` ou `PTHREAD_CREATE_DETACHED`
- **Stack Size** : Taille de la pile du thread
- **Scheduling Policy** : Politique d'ordonnancement
- **Priority** : Priorité du thread

6 Sémaphores POSIX

Qu'est-ce qu'un Sémaphore ?

Un **sémaphore** est un compteur qui contrôle l'accès à une ressource partagée. Il peut autoriser plusieurs threads simultanés (contrairement au mutex).

6.1 Fonctions Principales

```c
#include <semaphore.h>

sem_t semaphore;

// Initialisation : sem_init(sem, pshared, valeur_initiale)
// pshared = 0 : partagé entre threads du même processus
// valeur_initiale : nombre de ressources disponibles
sem_init(&semaphore, 0, 3);  // 3 ressources disponibles

// Attendre (décrémente le compteur, bloque si 0)
sem_wait(&semaphore);

// Section critique
// ...

// Signaler (incrémente le compteur)
sem_post(&semaphore);
```

```
18
19 // Destruction
20 sem_destroy(&semaphore);
```

Listing 9 – Utilisation des sémaphores

6.2 Exemple 7 : Pool de Connexions

```
1  #include <stdio.h>
2  #include <pthread.h>
3  #include <semaphore.h>
4  #include <unistd.h>
5
6  #define MAX_CONNEXIONS 3
7  #define NB_CLIENTS 10
8
9  // Sémaphore limitant à 3 connexions simultanées
10 sem_t pool_connexions;
11
12 void* client(void* arg) {
13     int id = *(int*)arg;
14
15     printf("Client %d : Demande de connexion...\n", id);
16
17     // Attendre qu'une connexion soit disponible
18     sem_wait(&pool_connexions);
19
20     printf("Client %d : CONNECT \n", id);
21
22     // Simulation d'utilisation de la connexion
23     sleep(2);
24
25     printf("Client %d : D connexion\n", id);
26
27     // Libérer la connexion
28     sem_post(&pool_connexions);
29
30     free(arg);
31     return NULL;
32 }
33
34 int main() {
35     pthread_t threads[NB_CLIENTS];
36
37     // Initialisation : maximum 3 connexions simultanées
38     sem_init(&pool_connexions, 0, MAX_CONNEXIONS);
39
40     // Création de 10 clients
41     for (int i = 0; i < NB_CLIENTS; i++) {
42         int* id = malloc(sizeof(int));
43         *id = i + 1;
44         pthread_create(&threads[i], NULL, client, id);
45         usleep(100000);  // Petit délai entre les créations
46     }
47
48     // Attente de tous les clients
49     for (int i = 0; i < NB_CLIENTS; i++) {
50         pthread_join(threads[i], NULL);
51     }
52
```

```
    sem_destroy(&pool_connexions);

    return 0;
}
```

Listing 10 – Limitation du nombre de connexions simultanées

7 Barrières de Synchronisation

Qu'est-ce qu'une Barrière ?

Une **barrière** est un point de synchronisation où tous les threads doivent arriver avant que l'un d'eux puisse continuer.

7.1 Exemple 8 : Calcul Parallèle par Phases

```c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define NB_THREADS 4

// Barrière pour synchroniser les threads
pthread_barrier_t barriere;

void* travailleur(void* arg) {
    int id = *(int*)arg;

    // Phase 1
    printf("Thread %d : Phase 1 en cours...\n", id);
    sleep(id);  // Temps variable selon le thread
    printf("Thread %d : Phase 1 termin e\n", id);

    // Attendre que tous les threads finissent la phase 1
    pthread_barrier_wait(&barriere);

    // Phase 2 (commence seulement quand tous ont fini phase 1)
    printf("Thread %d : Phase 2 en cours...\n", id);
    sleep(1);
    printf("Thread %d : Phase 2 termin e\n", id);

    // Attendre que tous finissent la phase 2
    pthread_barrier_wait(&barriere);

    printf("Thread %d : Toutes les phases termin es\n", id);

    free(arg);
    return NULL;
}

int main() {
    pthread_t threads[NB_THREADS];

    // Initialisation de la barrière pour 4 threads
    pthread_barrier_init(&barriere, NULL, NB_THREADS);

    for (int i = 0; i < NB_THREADS; i++) {
        int* id = malloc(sizeof(int));
```

```
43          *id = i + 1;
44          pthread_create(&threads[i], NULL, travailleur, id);
45      }
46
47      for (int i = 0; i < NB_THREADS; i++) {
48          pthread_join(threads[i], NULL);
49      }
50
51      pthread_barrier_destroy(&barriere);
52
53      return 0;
54  }
```

Listing 11 – Synchronisation avec barrière

8 Thread-Local Storage (TLS)

Variables Locales aux Threads

Le **TLS** permet à chaque thread d'avoir sa propre copie d'une variable globale.

8.1 Utilisation avec __thread

```
1  #include <stdio.h>
2  #include <pthread.h>
3
4  // Variable globale normale (partagée)
5  int compteur_global = 0;
6
7  // Variable thread-local (chaque thread a sa copie)
8  __thread int compteur_local = 0;
9
10 void* fonction(void* arg) {
11     int id = *(int*)arg;
12
13     for (int i = 0; i < 5; i++) {
14         compteur_global++;  // Partagé entre tous
15         compteur_local++;   // Propre à ce thread
16
17         printf("Thread %d : Global=%d, Local=%d\n",
18                id, compteur_global, compteur_local);
19     }
20
21     return NULL;
22 }
23
24 int main() {
25     pthread_t t1, t2;
26     int id1 = 1, id2 = 2;
27
28     pthread_create(&t1, NULL, fonction, &id1);
29     pthread_create(&t2, NULL, fonction, &id2);
30
31     pthread_join(t1, NULL);
32     pthread_join(t2, NULL);
33
34     return 0;
35 }
```

Listing 12 – Variables thread-local

9 Gestion des Erreurs

9.1 Codes d'Erreur Courants

Code	Signification
EAGAIN	Ressources système insuffisantes
EINVAL	Argument invalide
EDEADLK	Deadlock détecté
EBUSY	Ressource occupée
EPERM	Opération non permise

9.2 Exemple de Gestion d'Erreurs

```c
#include <stdio.h>
#include <pthread.h>
#include <errno.h>
#include <string.h>

void* fonction(void* arg) {
    return NULL;
}

int main() {
    pthread_t thread;
    int ret;

    // Création du thread avec vérification
    ret = pthread_create(&thread, NULL, fonction, NULL);
    if (ret != 0) {
        fprintf(stderr, "Erreur pthread_create: %s\n", strerror(ret));
        return 1;
    }

    // Join avec vérification
    ret = pthread_join(thread, NULL);
    if (ret != 0) {
        fprintf(stderr, "Erreur pthread_join: %s\n", strerror(ret));
        return 1;
    }

    return 0;
}
```

Listing 13 – Vérification des erreurs

10 Problèmes Classiques et Solutions

10.1 Deadlock (Interblocage)

Conditions du Deadlock

Un deadlock se produit quand :

1. Exclusion mutuelle
2. Hold and wait (détenir et attendre)
3. Pas de préemption
4. Attente circulaire

10.2 Exemple de Deadlock

```c
pthread_mutex_t mutex1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t mutex2 = PTHREAD_MUTEX_INITIALIZER;

void* thread1_func(void* arg) {
    pthread_mutex_lock(&mutex1);    // Thread 1 verrouille mutex1
    sleep(1);
    pthread_mutex_lock(&mutex2);    // Attend mutex2 (détenu par thread2)

    // Section critique

    pthread_mutex_unlock(&mutex2);
    pthread_mutex_unlock(&mutex1);
    return NULL;
}

void* thread2_func(void* arg) {
    pthread_mutex_lock(&mutex2);    // Thread 2 verrouille mutex2
    sleep(1);
    pthread_mutex_lock(&mutex1);    // Attend mutex1 (détenu par thread1)

    // Section critique

    pthread_mutex_unlock(&mutex1);
    pthread_mutex_unlock(&mutex2);
    return NULL;
}
```

Listing 14 – Situation de deadlock

10.3 Solution : Ordre de Verrouillage

```c
void* thread1_func(void* arg) {
    // Toujours verrouiller dans le même ordre
    pthread_mutex_lock(&mutex1);
    pthread_mutex_lock(&mutex2);

    // Section critique

    pthread_mutex_unlock(&mutex2);
    pthread_mutex_unlock(&mutex1);
    return NULL;
}

void* thread2_func(void* arg) {
    // Même ordre de verrouillage
```

```
15      pthread_mutex_lock(&mutex1);
16      pthread_mutex_lock(&mutex2);
17
18      // Section critique
19
20      pthread_mutex_unlock(&mutex2);
21      pthread_mutex_unlock(&mutex1);
22      return NULL;
23  }
```

Listing 15 – Éviter le deadlock avec un ordre fixe

11 Bonnes Pratiques

Règles d'Or

1. **Toujours** compiler avec `-pthread`
2. **Toujours** vérifier les valeurs de retour
3. **Minimiser** la taille des sections critiques
4. **Éviter** les variables globales quand possible
5. **Utiliser** des mutex pour protéger les données partagées
6. **Libérer** toujours les mutex verrouillés
7. **Détruire** les ressources (mutex, threads, etc.)
8. **Éviter** les deadlocks avec un ordre de verrouillage cohérent
9. **Utiliser** `pthread_join()` ou `pthread_detach()`
10. **Tester** avec différents nombres de threads

12 Résumé des Fonctions Principales

12.1 Tableau Récapitulatif

Fonction	Description
Gestion des Threads	
pthread_create()	Crée un nouveau thread
pthread_join()	Attend la terminaison d'un thread
pthread_exit()	Termine le thread appelant
pthread_detach()	Détache un thread
pthread_self()	Retourne l'ID du thread appelant
pthread_equal()	Compare deux IDs de threads
Mutex	
pthread_mutex_init()	Initialise un mutex
pthread_mutex_lock()	Verrouille un mutex
pthread_mutex_unlock()	Déverrouille un mutex
pthread_mutex_trylock()	Tente de verrouiller sans bloquer
pthread_mutex_destroy()	Détruit un mutex
Variables de Condition	
pthread_cond_init()	Initialise une variable de condition
pthread_cond_wait()	Attend un signal
pthread_cond_signal()	Réveille un thread
pthread_cond_broadcast()	Réveille tous les threads
pthread_cond_destroy()	Détruit une variable de condition
Sémaphores	
sem_init()	Initialise un sémaphore
sem_wait()	Décrémente (P operation)
sem_post()	Incrémente (V operation)
sem_destroy()	Détruit un sémaphore
Barrières	
pthread_barrier_init()	Initialise une barrière
pthread_barrier_wait()	Attend à la barrière
pthread_barrier_destroy()	Détruit une barrière

13 Exercices Types pour l'Examen

13.1 Exercice 1 : Analyse de Code

Question

Identifiez le problème dans ce code et proposez une correction :

```c
int compteur = 0;

void* incrementer(void* arg) {
    for (int i = 0; i < 100000; i++) {
        compteur++;
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, incrementer, NULL);
    pthread_create(&t2, NULL, incrementer, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("Compteur: %d\n", compteur);
    return 0;
}
```

Réponse : Race condition sur `compteur`. Solution : utiliser un mutex.

13.2 Exercice 2 : Implémentation

Question

Écrivez un programme avec 3 threads qui calculent la somme de parties différentes d'un tableau, puis combinent les résultats.

13.3 Exercice 3 : Synchronisation

Question

Implémentez le problème des lecteurs-rédacteurs : plusieurs lecteurs peuvent lire simultanément, mais un seul rédacteur à la fois, et pas de lecteur pendant l'écriture.

14 Conclusion

Points Clés à Retenir

- Les threads partagent la mémoire ⇒ besoin de synchronisation
- Mutex = exclusion mutuelle (1 seul thread à la fois)
- Variables de condition = attente d'événements
- Sémaphores = compteur de ressources
- Toujours protéger les sections critiques
- Attention aux deadlocks !
- Compiler avec `-pthread`

Bonne chance pour votre examen !